

QUALITY MONITORING FOR VEGETABLES AND FRUITS THROUGH IoT

ABSTRACT

In recent years, the integration of Internet of Things (IoT) technology into food quality monitoring systems has emerged as a pivotal innovation in ensuring the freshness and safety of produce. This mini-project focuses on leveraging IoT to monitor the quality of vegetables and fruits using an ethylene gas sensor. Ethylene, a byproduct of the decomposition of organic matter, can be an indicator of spoilage in fruits and vegetables. By employing a ethylene gas sensor, the system detects variations in ethylene levels that correlate with the deterioration of produce quality.

This approach not only enhances the ability to maintain high-quality standards but also provides actionable insights for managing inventory and reducing waste. The implementation of such a system aims to improve food safety, extend shelf life, and optimize supply chain efficiency. By integrating advanced sensor technology with IoT, this project seeks to offer a scalable and effective solution for modernizing food quality monitoring practices.

CONTENTS

Acknowledgements	5
Abstract	6
Contents	7
List Of Figures	10
Chapter 1 Overview	12
1.1 Introduction	12
1.2 Aim of the system	12
1.3 Methodology	12
1.4 Significance of work	13
1.5 Organization of work	13
1.6 Conclusion	14
Chapter 2 Introduction to Embedded Systems	15
2.1 Introduction	15
2.1.1 Characteristics of IoT	15
2.1.2 Sensors and Actuators	16
2.1.3 Classification of IoT	16
2.1.4 Applications of IoT	17
2.2 IoT Platforms and Computing	18
2.3 Advantages of IoT	20
2.4 Conclusion	21
Chapter 3 Hardware / Software Components	22
3.1 Introduction	22
3.2 DHT11 Sensor	22
3.2.1 Introduction	22

3.2.2 Working Principle	22
3.2.3 Circuit Connections	23
3.3. ESP8266	24
3.3.1 Introduction	24
3.3.2 Circuit Diagram of ESP8266 WIFI Module	25
3.3.3 Pin Description of ESP8266 WIFI Module	26
3.4 Gas Sensor MQ-4	28
3.4.1 Introduction	28
3.4.2 Working Principle	28
3.4.3. Circuit Connections	29
3.5 Arduino Uno	30
3.5.1 Introduction	30
3.5.2 Working principle	31
3.5.3 Applications	32
3.6 LCD display	32
3.6.1 Introduction	32
3.6.2 Working principle	33
3.6.3 Applications	33
3.7 Software	34
3.7.1 Introduction	34
3.7.2 Arduino IDE	34
3.7.3 Embedded Systems	35
3.7.4 Thing speak	35
3.8 Conclusion	37
Chapter 4 Project Description	38
4.1 Introduction	38
4.2 Block Diagram	38
4.3 Working	39
4.4 Code and Work Flow	40
4.5 Software Section	41

4.6 Circuit diagram	42
4.7 Code	42
Chapter 5 Result and Discussions	46
5.1 Observations and Graphs	46
5.2 Analysis of Sensor Readings	46
Chapter 6 Advantages and Applications	48
Bibliography	50
Appendix	51

LIST OF FIGURES

Fig.2.1 IoT Platform	19
Fig.2.2 IoT computing	20
Fig.3.1 DHT11 Sensor	24
Fig.3.2 ESP8266 Wifi Module	25
Fig.3.3 Block Diagram of a Configurable Logic block	26
Fig.3.4 Pin Diagram of ESP8266 Wifi Module	26
Fig.3.5 MQ4 Gas Sensor	28
Fig.3.6 Arduino Uno	30
Fig.3.7 16*2 Lcd display	33
Fig.3.8 Arduino IDE	35
Fig.4.1 Block Diagram of the Project	39
Fig 4.2 Workflow of the Project	41
Fig.4.3 Circuit Connection Diagram	42
Fig.5.1 Real time monitoring interface	47
Fig.5.2 Lcd display when food is spoiled	47
Fig.5.3 Lcd display when food is fresh	47

CHAPTER 1

INTRODUCTION

1.1 Motivation

Food quality monitoring plays an essential role in reducing food waste, enhancing food safety, and ensuring the highest standards of freshness during storage and transport. Traditional methods for assessing food quality often lack the capability to provide continuous, real-time data, which can result in delays in identifying spoilage. With the rise of Internet of Things (IoT) technologies, integrating sensors to monitor environmental and chemical conditions has emerged as a reliable and efficient solution.

This project harnesses the power of IoT through the integration of a gas sensor and the DHT11 temperature and humidity sensor, creating a comprehensive system to assess food quality. The gas sensor detects ethylene levels, a key indicator of produce spoilage, while the DHT11 sensor monitors ambient temperature and humidity, factors that significantly affect food freshness. This dual-sensor approach, combined with IoT capabilities, enables stakeholders to take proactive actions in response to real-time data, thus minimizing waste and extending the shelf life of fruits and vegetables.

1.2 Aim of the system

The primary objective of this system is to enable accurate and real-time monitoring of food spoilage indicators—namely, ethylene concentration, temperature, and humidity. By leveraging both chemical and environmental data, the system seeks to predict and prevent spoilage more effectively than traditional methods. Real-time alerts and monitoring capabilities allow for timely intervention, ensuring that produce is preserved under optimal conditions throughout its supply chain journey. This project thus aims to support industry stakeholders in minimizing food waste, improving safety, and optimizing inventory management practices.

1.3 Methodology

This system's design is structured around a seamless integration of hardware components, namely the Arduino uno, a gas sensor, and the DHT11 temperature and humidity sensor. Each component has a distinct role: the gas sensor measures ethylene levels associated with spoilage, while the DHT11 sensor provides critical temperature and humidity readings that influence freshness. Monitoring occurs continuously with sensor. The data is then processed and analyzed for any indications of spoilage or suboptimal conditions, enabling automated alerts when thresholds are breached.

Hardware Specifications:

- Arduino UNO
- MQ4 Gas Sensor
- DHT11 Temperature and Humidity sensor
- 16x2 LCD

Software Specifications:

- Arduino IDE
- Thing speak

1.4 Significance of the work

Ensuring food safety and quality is critical to reducing the incidence of foodborne illnesses and enhancing consumer confidence. By offering continuous monitoring, this system enables early detection of spoilage, significantly improving the ability to maintain safe, high-quality food in the market. The integration of gas and environmental sensors 2 within an IoT framework also supports optimized inventory management by allowing stakeholders to identify and prioritize at-risk produce, thus reducing waste and maximizing efficiency within the supply chain.

Furthermore, the combination of gas sensors with temperature and humidity monitoring facilitates a comprehensive understanding of food quality, as it captures both chemical markers and environmental influences on spoilage. This holistic approach enhances predictive accuracy,

allowing for precise control over storage and transport conditions, which ultimately contributes to safer, fresher, and higher-quality produce for consumers.

1.5 Organization of the work

Chapter 1 provides an overview and the goals of the project.

Chapter 2 introduces embedded systems and their relevance.

Chapter 3 discusses the hardware and software components used.

Chapter 4 elaborates on the project development, including circuit design and code.

Chapter 5 presents the experimental results and observations.

Chapter 6 concludes the work and outlines the future scope.

1.6 Conclusion

In summary, this project's development of an IoT-based food quality monitoring system, incorporating both a gas sensor and a DHT11 temperature and humidity sensor, represents a significant advancement in food safety technology. By capturing real-time data on ethylene levels, temperature, and humidity, the system provides a proactive solution to spoilage detection and management, enabling swift, data-driven interventions. This approach enhances the overall quality, safety, and lifespan of fruits and vegetables, offering a transformative impact on food preservation and supply chain management.

CHAPTER 2

INTRODUCTION TO EMBEDDED SYSTEMS

2.1 Introduction

The Internet of Things (IoT) has revolutionized various industries by enabling smart, interconnected devices to communicate and exchange data. In the realm of food quality monitoring, IoT plays a crucial role in ensuring the freshness and safety of vegetables and fruits. By integrating IoT technology with ethylene gas sensors, it is possible to continuously monitor the quality of produce, providing real-time data that can help in making informed decisions to prevent spoilage and ensure food safety.

2.1.1 Characteristics of IoT

1. Connectivity:

Connectivity is an essential feature of IoT. IoT lets you connect mobile phones, laptops, and other internet devices. Any person can get information about anything at any time and place. IoT can connect through several wireless devices, like sensors, mobile phones, trackers, etc. This way, the person will not have to wait for an internet connection to operate a device.

2. Identity of Things:

The collaboration of name and number gives an identity to an internet device. Giving an identity to the device is an essential aspect of IoT. Identity helps to differentiate between various internet devices and select the device we want to send the command. Every device needs a different controlling power based on the type of data provided. It is essential to give a unique identity to every device so that we can set up passwords or other security means. For example, fingerprints, face recognition IP addresses, and Face lock systems are several means of security given to the different identified devices to protect them.

3. Intelligence:

The intelligence of IoT devices depends on the sensors' intelligence. The sensors send the data to the user for further analysis. We need to update the IoT devices regularly to get the smart work done. It adds to their features and makes them smarter.

4. Scalability:

Scalability means the amount of data one can handle efficiently. The IoT has created a setup to handle enormous data and generate useful analysis.

2.1.2 Sensors and Actuators

Sensors: Sensors are devices or components that detect and respond to changes in the environment, converting physical parameters into signals that can be measured or recorded. They are integral to various applications, including industrial automation, environmental monitoring, healthcare, consumer electronics, and more.

Types of Sensors:

- **Temperature sensor:**
- Measure heat to detect temperature changes. Examples include thermocouples, thermistors, and resistance temperature detectors (RTDs).
- **Pressure Sensors:** Measure the force exerted by a fluid or gas. Common types are piezoelectric, capacitive, and strain gauge sensors.
- **Optical Sensors:** Detect light or changes in light. Photodiodes and phototransistors are common examples.
- **Motion Sensors:** Detect movement or acceleration. Examples are accelerometers and gyroscopes.
- **Humidity Sensors:** Measure the moisture content in the air. Capacitive and resistive humidity sensors are typical examples.
- **Gas Sensors:** Detects the presence of various gases. Examples include metal oxide sensors and electrochemical sensors.

Actuators:

Actuators are devices that convert energy into motion. They are essential components in various systems, enabling machines and devices to perform physical actions. Actuators are found in a wide range of applications, from industrial machinery to everyday consumer electronics, providing the means to control movement and operation.

Types of actuators:

There are several types of actuators, each suited to different applications and operating principles. Electric actuators use electrical energy to produce motion and are common in applications requiring precise control, such as robotics and automotive systems. Hydraulic actuators use fluid pressure to generate movement and are known for their high force and durability, making them ideal for heavy machinery and construction equipment. Pneumatic

actuators operate using compressed air and are often used in automation and control systems due to their simplicity and speed.

2.1.3 Classification of IoT:

5 Layer Architecture of IoT:

When project work is done with various cutting edge technologies and broad application areas, 5 layer architecture is considered as best. the 5 Layer model can be considered as an extension to the basic architecture of IoT because it has two additional layers to the basic model.

- **Perception Layer:**

This is the first layer of IoT architecture. In the perception layer, a number of sensors and actuators are used to gather useful information like temperature, moisture content, intruder detection, sounds, etc. The main function of this layer is to get information from surroundings and to pass data to another layer so that some actions can be done based on that information.

- **Network Layer:**

As the name suggests, it is the connecting layer between perception and middleware layer. It gets data from the perception layer and passes data to the middleware layer using networking technologies like 3G, 4G, UTMS, WiFi, infrared, etc. This is also called communication layer because it is responsible for communication between perception and middleware layer. All the transfer of data is done securely keeping the obtained data confidential.

- **Middleware Layer:**

Middleware Layer has some advanced features like storage, computation, processing, and action taking capabilities. It stores all data-set and based on the device address and name it gives appropriate data to that device. It can also make decisions based on calculations done on data-set obtained from sensors.

- **Application Layer:**

The application layer manages all application processes based on information obtained from the middleware layer. This application involves sending emails, activating alarms, security systems, turning on or off a device, smartwatch, smart agriculture, etc.

- **Business Layer:**

The success of any device does not depend only on technologies used in it but also how it is being delivered to its consumers. Business layer does these tasks for the device. It involves making flowcharts, graphs, analysis of results, and how devices can be improved, etc.

2.1.4 Applications of IoT

IoT applications are transforming various sectors by enhancing efficiency, convenience, and safety:

1. **Smart Homes:** IoT devices automate household tasks for improved convenience. Smart thermostats, lights, and security cameras are remotely controllable, while appliances like fridges and washing machines provide notifications, streamlining household management.

2. **Healthcare:** IoT in healthcare enables remote monitoring, reduces readmissions, and improves outcomes. Wearables track health data, and devices like insulin pumps and heart monitors supply real-time information for timely medical interventions.

3. **Industrial Automation:** IoT improves industrial efficiency by tracking machinery, inventory, and assets. Predictive maintenance anticipates failures, reducing downtime, and enhancing resource management.

4. **Agriculture:** IoT sensors support precision farming by providing data on soil, moisture, and temperature, helping optimize irrigation and fertilization. Drones aid in crop monitoring, improving agricultural efficiency.

5. **Smart Cities:** IoT in smart cities enhances urban living through smart traffic, parking, and waste management systems. Environmental sensors track air quality, fostering healthier, more efficient urban spaces.

2.2 IoT platforms and Computing

IoT (Internet of Things) platforms and computing involve the integration of hardware, software, and communication technologies to connect and manage devices, enabling them to communicate, process data, and make decisions. Here are some key aspects:

IoT platforms serve as the middleware between the hardware (sensors, devices) and application layers. They provide tools and services to manage, connect, and secure IoT devices, as well as to analyse and visualise the data they generate. Some key components include:-

- Device Management: Tools to monitor, maintain, and update IoT devices remotely.
- Connectivity Management: Services to manage the communication between devices and the cloud or other devices.
- Data Management: Solutions for data storage, processing, and analysis.
- Security: Measures to protect data integrity, privacy, and device security.

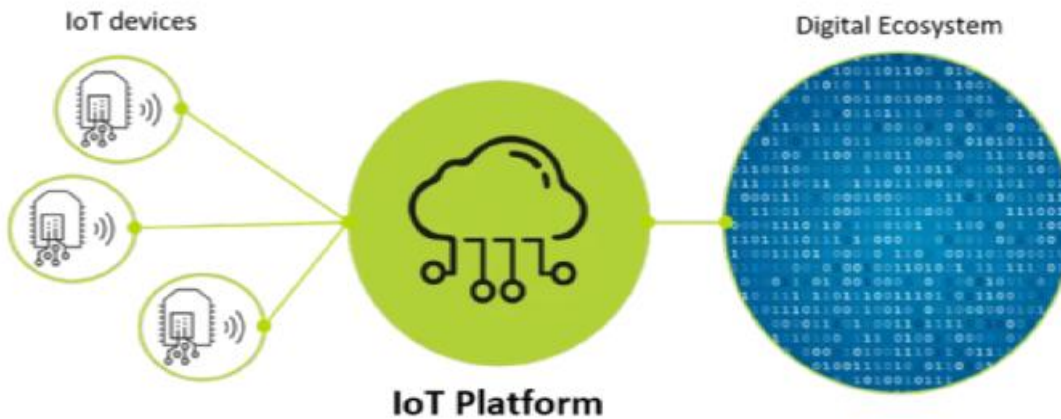


Fig 2.1: IoT Platform

IoT computing involves processing data from IoT devices, which can happen at different layers:

- **Edge Computing:** Data is processed locally on the device or nearby before being sent to the cloud. This reduces latency and bandwidth usage, making it ideal for time sensitive applications.
- **Fog Computing:** Extends cloud computing to the edge of the network, offering more processing power and storage closer to the devices than pure edge computing.
- **Cloud Computing:** Centralised data processing and storage in the cloud, providing scalability and powerful analytics capabilities.

Applications:

Smart Home: IoT devices like smart thermostats, lights, and security systems enhance home automation and security.

Industrial IoT (IIoT): Sensors and devices in manufacturing and industrial processes improve efficiency, predictive maintenance, and safety.

Healthcare: Wearables and remote monitoring devices provide real-time health data, improving patient care.

Smart Cities: IoT technologies manage traffic, energy consumption, and public services, making cities more efficient and livable.

Agriculture: IoT devices monitor soil conditions, weather, and crop health to optimize farming practices.

IoT platforms and computing are transforming various industries by enabling smarter, data-driven decisions and enhancing efficiency and productivity.

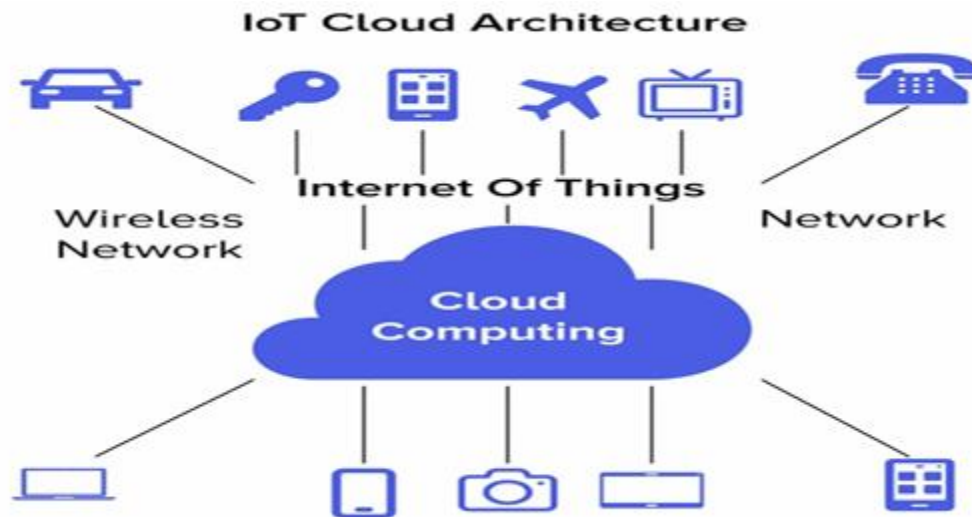


Fig.2: IoT Computing

2.3 Advantages of IoT

The Internet of Things (IoT) represents a significant technological advancement that is reshaping industries and transforming everyday life. By connecting a vast array of devices and enabling them to communicate and share data, IoT is unlocking unprecedented opportunities for efficiency, productivity, and innovation. The following sections outline the key advantages of IoT and how they are driving positive change across various sectors.

Increased Efficiency: One of the most significant advantages of IoT is the ability to improve efficiency in various processes. By collecting and analyzing data from connected devices, organizations can streamline operations, reduce waste, and optimize resource use. For example, in manufacturing, IoT sensors can monitor equipment performance in real time, allowing for predictive maintenance that minimizes downtime and extends machinery lifespan.

Enhanced Productivity: IoT technology can automate routine tasks, freeing up human resources for more complex and creative work. In agriculture, IoT devices can monitor soil moisture and weather conditions, automatically adjusting irrigation systems to ensure optimal crop growth. Similarly, in smart homes, IoT devices can manage lighting, heating, and security systems, improving convenience and comfort for residents.

Enhanced Customer Experience: IoT enhances customer experiences by providing personalized and responsive services. In healthcare, wearable IoT devices can monitor patients' vital signs and send alerts to medical professionals if any anomalies are detected, enabling timely intervention. In the automotive industry, connected cars can offer personalized navigation, entertainment options, and proactive maintenance alerts, improving driver safety and satisfaction.

Cost Savings: IoT can lead to significant cost savings by optimizing operations and reducing waste. Smart energy management systems in buildings can monitor and control energy usage, reducing utility bills. In logistics, IoT-enabled fleet management systems can optimize routes, monitor vehicle conditions, and reduce fuel consumption, leading to lower operational costs.

2.4 Conclusion

In conclusion, IoT transforms device interaction by enabling automation, efficiency, and real-time monitoring. When combined with sensors and actuators, IoT offers robust solutions for continuous data analysis and swift response, making it essential for food quality monitoring to preserve freshness and ensure consumer safety.

CHAPTER 3

HARDWARE/SOFTWARE DESCRIPTION

3.1 introduction

Monitoring the quality of fruits and vegetables is important to make sure they are fresh and safe to eat. Instead of checking them by hand, we can use machines and computer programs to do this job faster and more accurately.

The hardware includes tools like sensors to check things like color, size, and smell. The software uses this information to decide if the fruits or vegetables are good or bad. This system helps farmers, sellers, and buyers make better choices and reduce waste.

3.2 DHT11 Sensor

3.2.1 Introduction

The **DHT11** sensor is used to measure **temperature and humidity**. It has a built-in thermistor and a humidity sensing component. It sends digital signals to microcontrollers, making it easy to use in electronics projects. The sensor is simple to connect, and suitable for basic weather and environmental monitoring systems.

The **DHT11 sensor** measures **temperature** using a built-in **thermistor**. It can measure temperatures from **0°C to 50°C** with an accuracy of about $\pm 2^\circ\text{C}$. The temperature data is sent in **digital format** to a microcontroller like Arduino or Raspberry Pi, making it easy to use in small weather and monitoring projects.

3.2.2 Working Principle

These components detect changes in temperature and moisture in the air. The sensor has a small chip inside that converts these readings into a **digital signal** using an **ADC (Analog to Digital Converter)**.

This digital signal is then sent to a microcontroller (like Arduino) through a **single data pin**. The sensor sends data every **1 second**, and the communication follows a **one-wire protocol**, which makes it easy to use with many devices.

The working process of the DHT11 sensor follows these steps:

1. **Humidity Sensor:**

It uses a **capacitive humidity sensor**. When the moisture in the air changes, the **capacitance** of the sensor also changes. This change is measured and converted into a humidity value.

2. **Temperature Sensor:**

It uses an **NTC thermistor** (a resistor that changes resistance with temperature). The sensor measures this resistance change to calculate the temperature.

3. **Signal Conversion:**

An internal chip inside the DHT11 processes both the temperature and humidity signals and converts them into a **digital output**.

4. **Data Transmission:**

The processed data is sent to a microcontroller (like Arduino) through a **single digital pin** using a **1-wire communication protocol**.

This efficient and straightforward operation allows the DHT11 to deliver reliable data with minimal latency, enabling IoT applications to make real-time adjustments based on environmental conditions. The sensor's built-in error-checking protocol ensures that the data transmitted is accurate, making it a dependable choice for food quality monitoring systems.

3.2.3 Circuit Connections

Integrating the DHT11 sensor into an IoT monitoring system involves connecting it to a microcontroller, such as the ESP8266, which receives and processes the data. The DHT11 has four pins: VCC, GND, Data, and NC (not connected). The following steps outline the circuit connections required to integrate the DHT11 sensor effectively:

1. **Power Connections:**

- Connect the VCC pin of the DHT11 sensor to a 3.3V or 5V power source, depending on the power requirements of the microcontroller. Typically, 3.3V is sufficient and helps ensure compatibility with low-power IoT devices.
- Connect the GND (ground) pin to the ground (GND) pin on the microcontroller to establish a common reference for power and signal communication.

2. **Data Line Connection:**

- The Data pin transmits the digital signal generated by the DHT11 sensor. Connect this pin to a digital input pin on the microcontroller (e.g., GPIO4 or GPIO5 on the ESP8266).

3. **Single-Wire Communication Protocol:**

- The DHT11 uses a single-wire communication protocol, meaning data transmission and power flow occur through the same wire. The microcontroller communicates with the sensor by sending a start signal, prompting the DHT11 to send back a 40-bit data packet containing the temperature and humidity readings.
- Once the data packet is received, the microcontroller parses the information into separate values for temperature and humidity, applying necessary conversions based on the data format to produce human-readable outputs.

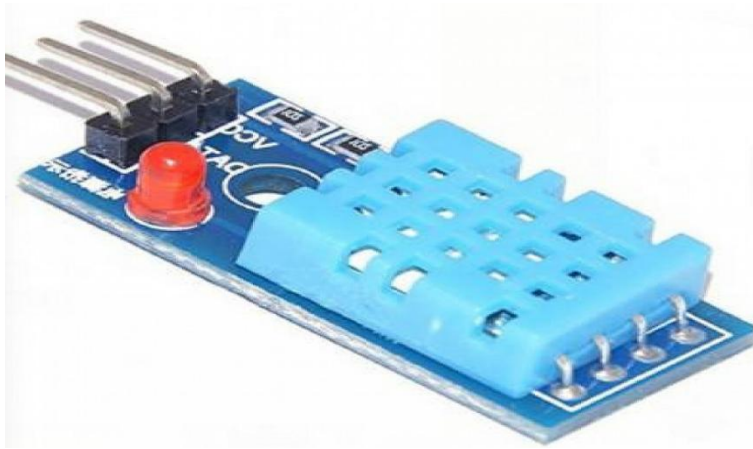


Figure 3.1 DHT11 Sensor

3.3 ESP-8266

3.3.1 Introduction

Most people call ESP8266 a WIFI module, but it is actually a microcontroller. ESP8266 is the name of the microcontroller developed by Espressif Systems which is a company based out of shanghai. This microcontroller has the ability to perform WIFI related activities hence it is widely used as a WIFI module. There are many types of ESP8266 modules available ranging from ESP8266-01 to ESP8266-12. The one that we are using in the tutorial is the ESP8266-01 because it is the cheapest one and easily available. However all the ESP modules have only one type of ESP processor, what differs is only the type of breakout board used. The breakout board of ESP8266-01 will have only 2 GPIO pins whereas in other boards it will be higher.



Figure 3.2:ESP8266 Wi-Fi Module

3.3.2 Circuit Diagram of ESP-8266 Wi-Fi module:

An ESP8266 is a highly integrated Wi-Fi chip that can be used to build IoT projects. The typical circuit diagram of an ESP8266 involves several key components and connections to ensure proper functionality. At the heart of the circuit is the ESP8266 module itself, which requires a stable power supply of 3.3V. This power is often provided by a voltage regulator, such as the AMS1117, which steps down the voltage from a 5V power source to the required 3.3V.

The ESP8266 module has several GPIO pins that can be used to interface with sensors, actuators, and other peripherals. For example, a digital sensor can be connected to one of the GPIO pins, with the sensor's output pin connected to the GPIO pin and its VCC and GND pins connected to the 3.3V and GND, respectively. Pull-up or pull-down resistors may be added to the GPIO pins to ensure proper logic levels.

For communication, the ESP8266 uses UART (Universal Asynchronous Receiver/Transmitter). The UART RX (receive) and TX (transmit) pins are connected to corresponding pins on a USB-to-TTL serial adapter for programming and debugging purposes. The TX pin of the ESP8266 is connected to the RX pin of the adapter, and the RXpin of the ESP8266 is connected to the TX pin of the adapter.

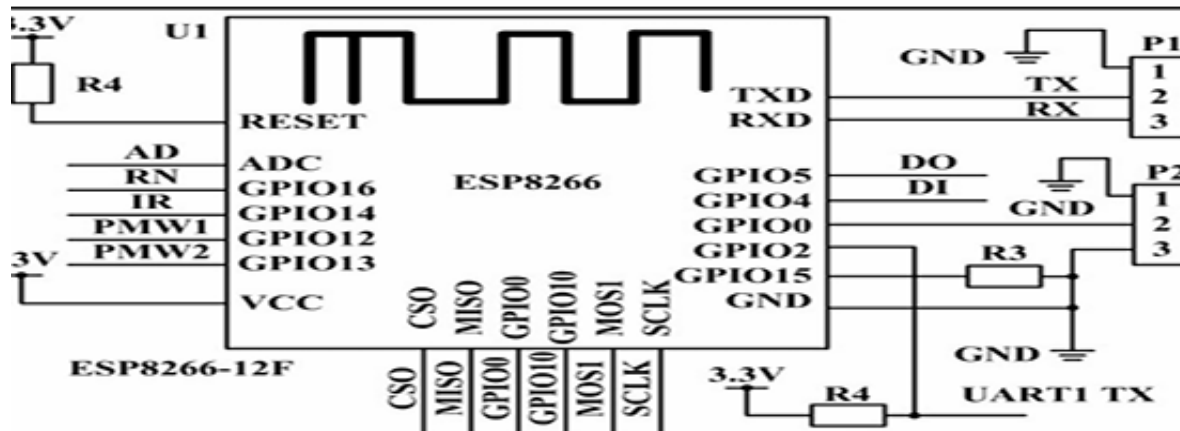


Figure 3.3: Block diagram of a configurable logic block

3.3.3 ESP-8266 WIFI MODULE:

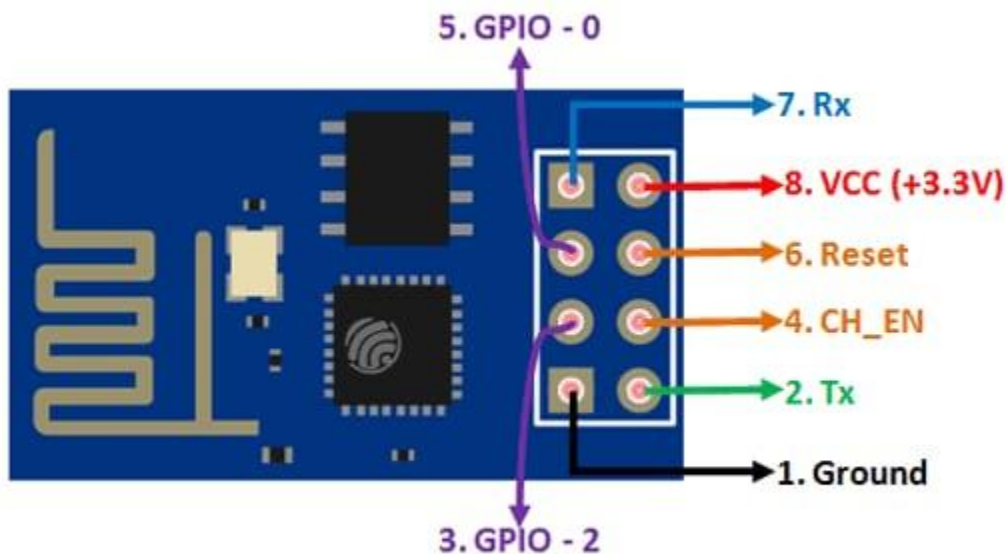


Figure 3.4: Pin Diagram of ESP-8266 WIFI Module

The NodeMCUESP8266 is an open-source, low-cost microcontroller development board. Here's a brief pin description for the NodeMCU ESP8266:

3V3 (3.3 V Power Pin): This pin supplies a stable 3.3 V voltage, essential for powering the board and connected peripherals.

GND (Ground Pin): Ground reference point for electrical circuits. Connect your ground wires here.

RST (Reset Pin): An active low pin that resets the microcontroller when pulled low. Useful for restarting the board or clearing any glitches.

EN (Enable Pin): An active high pin that enables or disables the entire board. Pulling it high activates the NodeMCU.

TX (Transmit Pin): Part of the UART (Universal Asynchronous Receiver-Transmitter) interface. It sends serial data from the NodeMCU to other devices.

RX (Receive Pin): Also, part of the UART interface. It receives serial data from external devices and forwards it to the NodeMCU.

GPIO0 and GPIO2 (General-Purpose I/O Pins): These pins serve as flexible digital input/output pins. You can configure them for various tasks, such as reading sensors, controlling actuators, or interfacing with other components.

3.2.4 FEATURES OF ESP-8266

The ESP8266 WiFi module is a compact, versatile component widely used in IoT applications for its built-in WiFi and microcontroller capabilities. It connects devices to WiFi networks, supporting wireless data transmission and internet access. Known for low power consumption and multiple GPIO pins, the ESP8266 can interface with sensors, actuators, and peripherals, making it ideal for various IoT projects. Supported by a rich development ecosystem with Arduino IDE compatibility, OTA updates, and affordable pricing, it simplifies development while delivering reliable performance for applications in remote monitoring, data acquisition, and control over WiFi networks.

Key features:

- **Integrated WiFi:** Connects directly to WiFi networks, enabling wireless communication.
- **Microcontroller Capabilities:** Built-in microcontroller (Xtensa LX106) can handle various tasks and interact with sensors and actuators.
- **Low Power Consumption:** Efficient design suitable for battery-powered IoT applications.
- **Compact Size:** Small form factor for easy integration into space-limited projects.
- **Extensive GPIO Pins:** Allows flexible interfacing with external devices.
- **Development Ecosystem:** Extensive support through documentation, forums, libraries, and resources.

- **OTA Updates:** Supports remote firmware updates, simplifying device maintenance.
- **Arduino IDE Compatibility:** Programmable via Arduino IDE, with support for other environments.

3.4 GAS SENSOR-MQ4

3.4.1 Introduction

The MQ-4 gas sensor is a widely used semiconductor gas sensor that is designed to detect methane (CH_4) and other combustible gases such as LPG, butane, propane, and hydrogen. It is commonly used in gas leakage detection systems for industrial and domestic safety.

The MQ4 uses a tin dioxide (SnO_2) semiconductor layer to sense gases, with conductivity increasing as gas concentration rises. This versatile, sensitive sensor is commonly used in gas detection for home automation, industrial safety, and environmental monitoring. Compact, affordable, and easy to integrate with microcontrollers like Arduino, the MQ4 plays a key role in alerting users to hazardous gases, supporting proactive safety measures.

MQ4 Pinout



Figure 3.5:MQ4 Gas Sensor

3.4.2 Working Principle

The MQ-4 sensor works on the principle of change in resistance of the SnO_2 (tin dioxide) sensing material. In clean air, the resistance is high. When the sensor detects a combustible gas, the gas

molecules interact with the sensor surface, reducing the resistance. This change is converted to an analog voltage which can be read by a microcontroller like Arduino, Raspberry Pi, etc.

Applications:

MQ4 gas sensors find applications in various fields including industrial safety, domestic gas leakage detection, air quality monitoring, and fire detection systems. They play a crucial role in alerting users to the presence of potentially harmful gases, enabling timely responses to ensure safety and environmental protection.

3.4.3 Circuit Connections

Integrating a gas sensor like the MQ2 into an IoT monitoring system involves connecting it to a microcontroller, such as the ESP8266, which reads and processes gas concentration data. The MQ2 sensor typically has four pins: VCC, GND, Digital Out (D0), and Analog Out (A0). The following steps outline the circuit connections required to effectively integrate the MQ2 gas sensor:

Power Connections:

- Connect the VCC pin of the MQ2 sensor to a 5V power source. While the ESP8266 operates at 3.3V, many gas sensors require 5V for optimal performance.
- Connect the GND (ground) pin on the sensor to the ground (GND) pin of the microcontroller to establish a common ground reference for reliable communication.

Signal Connections:

- Analog Output (A0): The analog output pin on the MQ4 provides a voltage proportional to the gas concentration. Connect this pin to an analog input on the microcontroller if available (e.g., A0 on ESP8266-based boards) to enable real-time analog data acquisition.
- Digital Output (D0): The digital output pin on the MQ4 provides a HIGH or LOW signal based on a preset threshold of gas concentration, determined by adjusting the onboard potentiometer. Connect this pin to a digital input on the microcontroller (e.g., GPIO4 or GPIO5 on the ESP8266). This setup is useful if you only need to detect when gas concentration exceeds a certain level, rather than a precise measurement.

Pull-Up Resistor (if needed):

- If required, add a pull-up resistor (typically 10kΩ) between the D0 pin and VCC to stabilize the digital signal. This helps maintain a steady high-level output when no gas is detected.

Heater and Warm-Up Period:

- The MQ2 sensor includes a built-in heating element that requires a warm-up period (usually 20 seconds to 2 minutes) to stabilize the sensor. The warm-up ensures that the sensor's measurements are accurate, especially when monitoring sensitive gas concentrations.

Analog Signal Processing (Optional):

- For more precise gas measurements, the analog output (A0) voltage can be processed through the microcontroller's analog-to-digital converter (ADC). The microcontroller converts the analog signal to a digital value, which can be calibrated to estimate gas concentration in parts per million (ppm) based on the sensor's specifications and calibration.

3.5 ARDUINO UNO

3.5.1 Introduction:



Fig:3.6:Arduino Uno

The **Arduino Uno** is one of the most popular microcontroller development boards in the Arduino family. It is an open-source electronics platform based on easy-to-use hardware and software. The

Arduino Uno board is based on the **ATmega328P** microcontroller and features a simple, user-friendly environment that makes it suitable for beginners and professionals alike.

Arduino Uno provides a versatile platform for building a variety of interactive projects, including home automation systems, robotics, sensor applications, and much more. The board can be programmed using the **Arduino IDE (Integrated Development Environment)**, which allows users to write and upload code to the board.

Key features of the Arduino Uno include:

- 14 digital input/output pins
- 6 analog inputs
- 16 MHz clock speed
- USB connection for programming and power
- Supports 5V logic

3.5.2 Working Principle:

The Arduino Uno works by interfacing the ATmega328P microcontroller with external components, such as sensors, actuators, and communication modules. The board is programmed using the Arduino IDE, where the user writes code (sketch) that is uploaded to the microcontroller.

The main working steps are:

Power Supply:

The Arduino Uno can be powered through a USB connection or an external power source (7-12V) via the power jack. The onboard voltage regulator provides a stable 5V output to power both the board and external connected components.

Input and Output Pins:

The board has 14 digital pins, which can either read input (such as from a button or a sensor) or send output (such as to control LEDs or motors). The 6 analog pins are used to read signals from analog sensors (e.g., temperature sensors, potentiometers) and convert them into digital values for processing.

Microcontroller:

The ATmega328P microcontroller interprets the code uploaded through the IDE and controls the digital and analog pins accordingly. It runs the code in a loop and reacts to input signals, processing them and providing output based on predefined conditions.

Programming the Board:

Code (called "sketch") is written in the Arduino IDE using a simplified version of C/C++. Once the sketch is written, it is uploaded to the Arduino board via a USB cable. The code runs on the microcontroller and can interact with external hardware such as sensors or actuators.

Communication:

Arduino Uno can communicate with other devices using serial communication (via USB or UART pins). It can also be expanded to support wireless communication modules (e.g., Wi-Fi or Bluetooth).

3.5.3 Applications:

- **Home automation:** Controlling appliances or lighting systems.
- **Robotics:** Building robots and autonomous systems.
- **Environmental monitoring:** Using sensors to collect data on temperature, humidity, and air quality.
- **Prototyping:** Creating prototypes of electronic products and systems.

3.6 LCD DISPLAY

3.6.1: Introduction

A 16×2 LCD is a type of alphanumeric display that can show 16 characters per line on 2 lines. It is commonly used in embedded systems and microcontroller-based projects for displaying sensor data, user interfaces, or messages. It is based on the HD44780 controller and is easy to interface with Arduino, Raspberry Pi, and other microcontrollers.

3.6.2: Working Principle:

The 16×2 LCD works on the principle of liquid crystal technology. Liquid crystals do not emit light directly but use a backlight to display characters. The display consists of two registers:

Command Register: Sends instructions like initializing, clearing display, setting cursor position.

Data Register: Stores data to be displayed. Characters are displayed using a dot matrix (usually 5x8 pixels per character). It uses ASCII values to show text and can be operated in 4-bit or 8-bit mode.

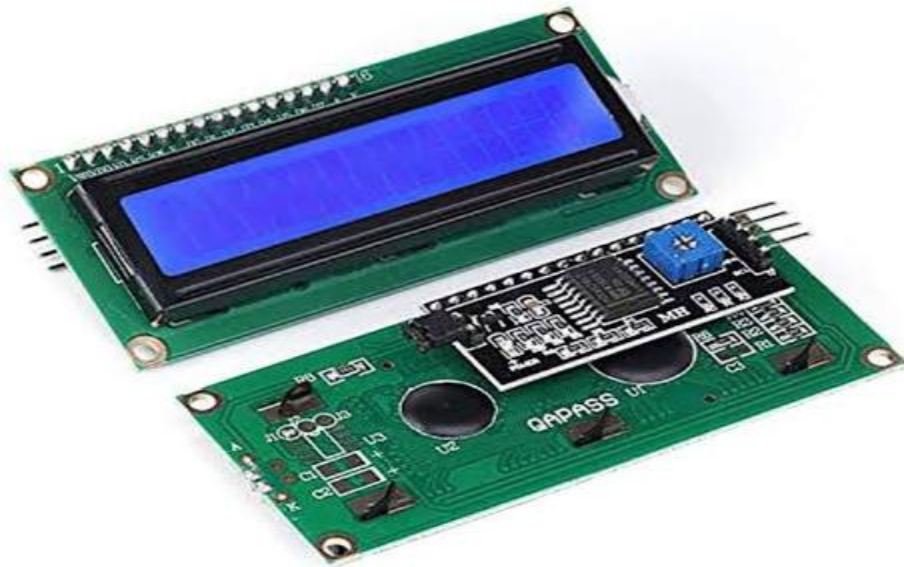


Fig 3.7: 16*2 LCD display

3.6.3: Applications:

- Digital thermometers and hygrometers
- IoT devices and weather monitoring systems
- Home automation interfaces
- Electronic voting machines
- Industrial process display

3.7 Software

3.7.1 Introduction

Quality Monitoring Software for Fruits and Vegetables With the increasing demand for fresh and high-quality produce, ensuring the quality of fruits and vegetables throughout the supply chain has become crucial. Quality monitoring software plays a vital role in automating the assessment and tracking of produce freshness, appearance, and safety from farm to fork.

This software leverages computer vision, machine learning, and sensor technologies to analyse various parameters such as color, texture, shape, ripeness, spoilage, and firmness. It provides real-time insights and alerts to farmers, distributors, and retailers to maintain consistent quality and reduce wastage.

3.7.2 ARDUINO IDE

The Arduino IDE is an open-source software, which is used to write and upload code to the Arduino boards. The IDE application is suitable for different operating systems such as Windows, Mac OS X, and Linux. It supports the programming languages C and C++. Here, IDE stands for one in Italian.

The program or code written in the Arduino uno is often called sketching. We need to connect the Genuine and Arduino board with the IDE to upload the sketch written in the Arduino IDE software. The sketch is saved with the extension .

The Arduino IDE will appear as:

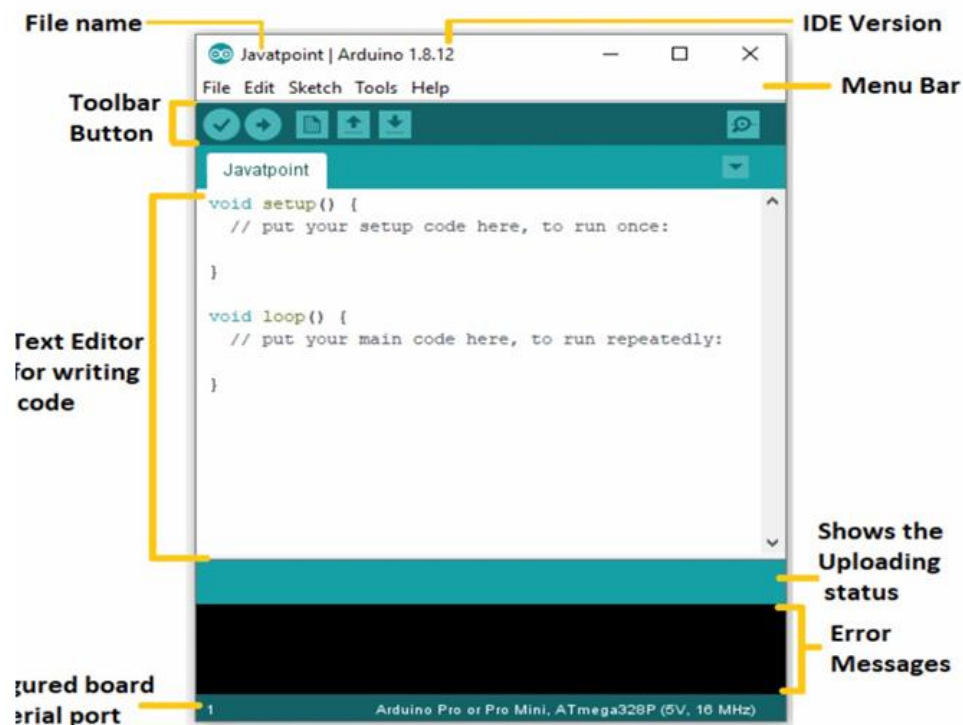


Figure 3.8: ARDUINO IDE

3.7.3 EMBEDDED SYSTEM

Embedded systems are specialized computing systems dedicated to specific tasks within larger electronic or mechanical devices. Unlike general-purpose computers, which are versatile and capable of running multiple applications, embedded systems are designed to perform predefined functions efficiently and reliably. They are typically found in everyday devices such as smartphones, household appliances, automotive electronics, and industrial machinery, where they handle tasks like control, monitoring, and data processing in real-time. These systems are characterised by their close integration with hardware components like sensors and actuators, enabling them to interact directly with the physical world. Due to their application-specific nature, embedded systems often operate with limited resources, requiring optimization in terms of power consumption, memory usage, and processing speed. Engineers focus on ensuring these systems are robust, secure, and compliant with industry standards, as they play critical roles in ensuring the functionality, safety, and efficiency of various technologies we rely on daily.

3.7.4 Thing Speak

1. Introduction

Thing Speak is an open-source Internet of Things (IoT) platform that enables the collection, storage, analysis, and visualization of data from sensors and devices over the Internet. It is widely used in embedded systems and IoT applications for real-time monitoring, data logging, and triggering events. Developed by Math Works, Thing Speak integrates seamlessly with MATLAB for advanced data analysis and visualization.

Thing Speak allows users to create **channels** to store data fields, and each channel can support multiple fields (up to 8 fields of data per channel). Devices like Arduino, ESP8266, ESP32, Raspberry Pi, etc., can be programmed to send data to ThingSpeak over Wi-Fi or the Internet using HTTP requests.

2. Working Principle

The working of Thing Speak can be broken down into several steps:

Step 1: Device Setup

IoT devices such as microcontrollers (ESP8266/ESP32) are equipped with sensors (e.g., temperature, humidity, gas sensors). These devices collect real-time data from their environment.

Step 2: Data Transmission

The device connects to a Wi-Fi network and sends sensor data to the ThingSpeak server via HTTP GET or POST requests. This requires the use of a **channel write API key**, which authenticates the device with the specific channel on ThingSpeak.

Step 3: Data Storage

ThingSpeak stores the incoming data in a structured format within its cloud-based database. Each channel holds entries for up to 8 fields, timestamps, and optional location data (latitude, longitude, elevation).

Step 4: Data Visualization

Users can visualize their data using built-in plotting tools on the ThingSpeak dashboard. The platform supports line graphs, bar graphs, gauges, and more.

Step 5: Data Analysis and Action

Using the MATLAB analytics tool embedded in ThingSpeak, users can run scripts to analyze data, perform predictions, or trigger actions (like alerts or device responses) based on logic or thresholds

3.8 Conclusion

The integration of both software and hardware components in quality monitoring systems for fruits and vegetables offers a comprehensive solution to ensure freshness, safety, and consistency in produce. Hardware components such as cameras, sensors (for temperature, humidity, gas levels, etc.), and microcontrollers enable real-time data collection from the environment and the produce itself. Software modules, powered by image processing, machine learning, and data analytics, analyze this data to detect ripeness, spoilage, and other quality parameters with high accuracy.

Together, these technologies automate the inspection process, reduce human dependency, and minimize post-harvest losses. This holistic approach not only improves efficiency in the supply chain but also contributes to food safety, better inventory management, and increased customer satisfaction. As agriculture moves towards digitization and automation, such smart systems will become essential for sustainable and scalable food production.

CHAPTER 4

PROJECT DESCRIPTION

4.1 Introduction

The Quality Food Monitoring for Vegetables and Fruits Using IoT project aims to enhance the quality, safety, and shelf-life of perishable produce by leveraging Internet of Things (IoT) technology. Traditional methods for assessing food spoilage are often reactive and limited to visual inspections, which can be inefficient and prone to error. This project addresses the need for a proactive, data-driven solution by monitoring ethylene gas levels (an indicator of ripening and spoilage) alongside environmental factors such as temperature and humidity. Real-time data collected from these parameters offers actionable insights into produce freshness, enabling timely interventions to minimize food waste, optimize inventory management, and uphold food safety standards.

4.2 Block Diagram

The block diagram represents the structural layout and communication flow within the IoT-based monitoring system. Key components include:

- 1. ESP8266 Microcontroller:** This is the primary control unit, handling data acquisition, processing, and Wi-Fi communication. It gathers data from the sensors and transmits it to the cloud for remote access.
- 2. MQ4 Gas Sensor:** The gas sensor specifically detects ethylene—a byproduct of produce decomposition—indicating spoilage levels. As the ethylene level rises, it suggests increased ripening or decay, triggering alerts.
- 3. DHT11 Temperature and Humidity Sensor:** Environmental conditions like temperature and humidity significantly impact produce freshness. The DHT11 sensor continuously monitors these parameters, providing data essential for maintaining optimal storage conditions.
- 4. Power Supply:** Ensures stable power for all connected components, essential for uninterrupted operation.

The flow of data starts at the sensors, which send readings to the ESP8266 microcontroller. The microcontroller processes this information and uploads it to the cloud, where it can be monitored by users in real time.

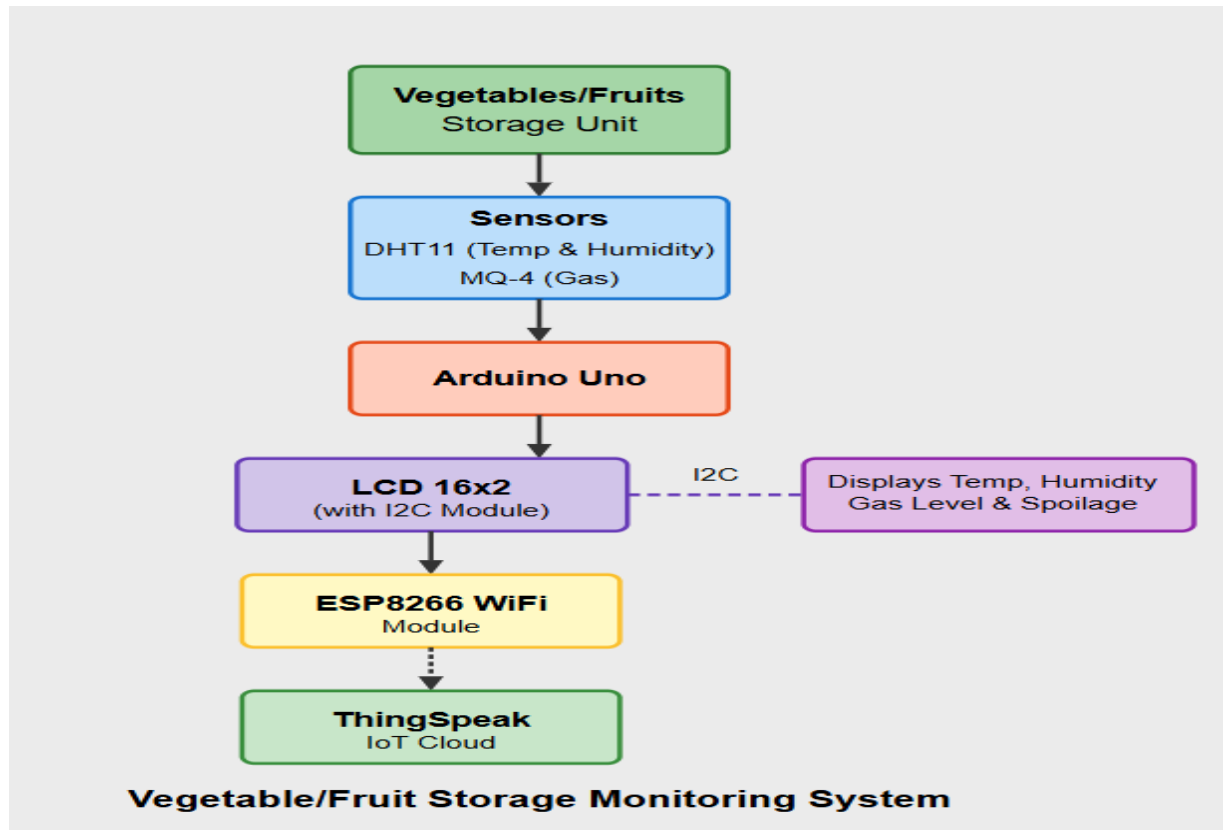


Fig 4.1: Block Diagram of the project

4.3 Working

The system's operation is based on continuous monitoring and real-time data transmission: 1. **Data Collection:** The MQ4 gas sensor detects ethylene concentrations, while the DHT11 measures temperature and humidity. These readings are collected at regular intervals by the ESP8266 microcontroller. 2. **Data Processing:** The microcontroller processes the sensor data and checks it against preset thresholds. For instance, if ethylene levels exceed the safe limit or if temperature/humidity values fall outside optimal ranges, the system flags the data as indicative of spoilage risk.

3. **Data Transmission:** Processed data is sent to the cloud via the Wi-Fi-enabled ESP8266 microcontroller. This enables remote access, allowing stakeholders to monitor storage conditions from anywhere in real time.

4. **Decision-Making and Action:** The real-time insights provided by the system enable stakeholders to make informed decisions regarding inventory management, reduce spoilage, and ensure only fresh produce is sent to market.

4.4 Workflow

The project's code is developed in the Arduino UNO, which is compatible with the ESP8266 microcontroller and facilitates easy integration with the sensors. The code execution and workflow follow this sequence:

1. **Sensor Initialization and Calibration:** The code initializes the MQ4 and DHT11 sensors, ensuring they are ready for data collection. Calibration routines are run, particularly for the gas sensor, to establish a baseline for ethylene concentration.

2. **Data Acquisition:** The sensors collect real-time data, which is then read by the ESP8266 microcontroller. Analog and digital values from the MQ4 sensor indicate ethylene levels, while the DHT22 provides digital readings for temperature and humidity.

3. **Threshold Comparison and Alert Generation:** The microcontroller compares the sensor readings to predetermined thresholds. If a reading indicates spoilage risk (e.g., high ethylene levels or suboptimal temperature/humidity), the system triggers an alert.

4. **Data Transmission to Cloud:** Using the integrated Wi-Fi capability of the ESP8266, data is sent to IoT platform. Here, it is processed for real-time visualization and storage, enabling historical trend analysis and generating actionable insights.

This workflow ensures an uninterrupted cycle of data collection, processing, and transmission, allowing users to monitor produce quality with minimal manual intervention.

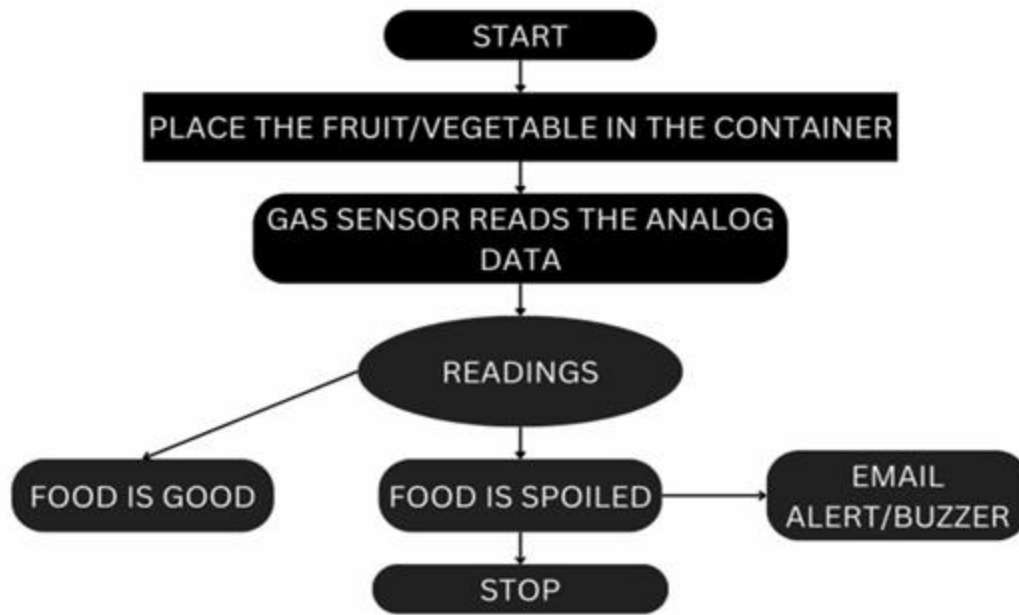


Fig 4.2 : Workflow of the project

4.5 Software Section

The software component of this project comprises three main platforms that facilitate sensor communication, data processing, remote monitoring, and control:

1. **Arduino UNO:** This open-source software provides an environment for writing, testing, and uploading code to the ESP8266 microcontroller. It supports the integration of the MQ4 and DHT11 sensors, enabling the ESP8266 to collect and transmit sensor data. Arduino libraries streamline the coding process for sensor readings, Wi-Fi connectivity.
2. **Embedded System:** Embedded systems integrate software directly into the hardware, ensuring that the sensors and microcontroller function seamlessly as a single system. This setup is designed for reliability and low power consumption, essential for continuous monitoring over extended periods.

By integrating these software platforms, the IoT-based monitoring system ensures effective communication between hardware components, data accessibility for users, and adaptability for various use cases in produce monitoring and quality control. This comprehensive setup ultimately enhances food safety, minimizes waste, and streamlines inventory management for perishable goods.

3. Thing Speak Platform:

Thing Speak is a cloud-based IoT analytics platform that enables real-time data collection, visualization, and analysis from connected devices. It acts as a bridge between sensors and users by providing an interface to send, store, and monitor data over the internet.

By integrating Thing Speak into an embedded system, sensor data can be seamlessly transmitted from the microcontroller to the cloud, allowing for remote access and continuous monitoring. This platform supports reliable communication, historical data tracking, and real-time graphing, making it ideal for IoT applications.

Thing Speak enhances the efficiency of monitoring systems by providing immediate insights into environmental conditions, supporting data-driven decision-making, and enabling automated responses. Its flexibility and ease of use make it well-suited for applications such as agriculture, smart homes, health monitoring, and industrial automation.

4.6 CIRCUIT DIAGRAM:

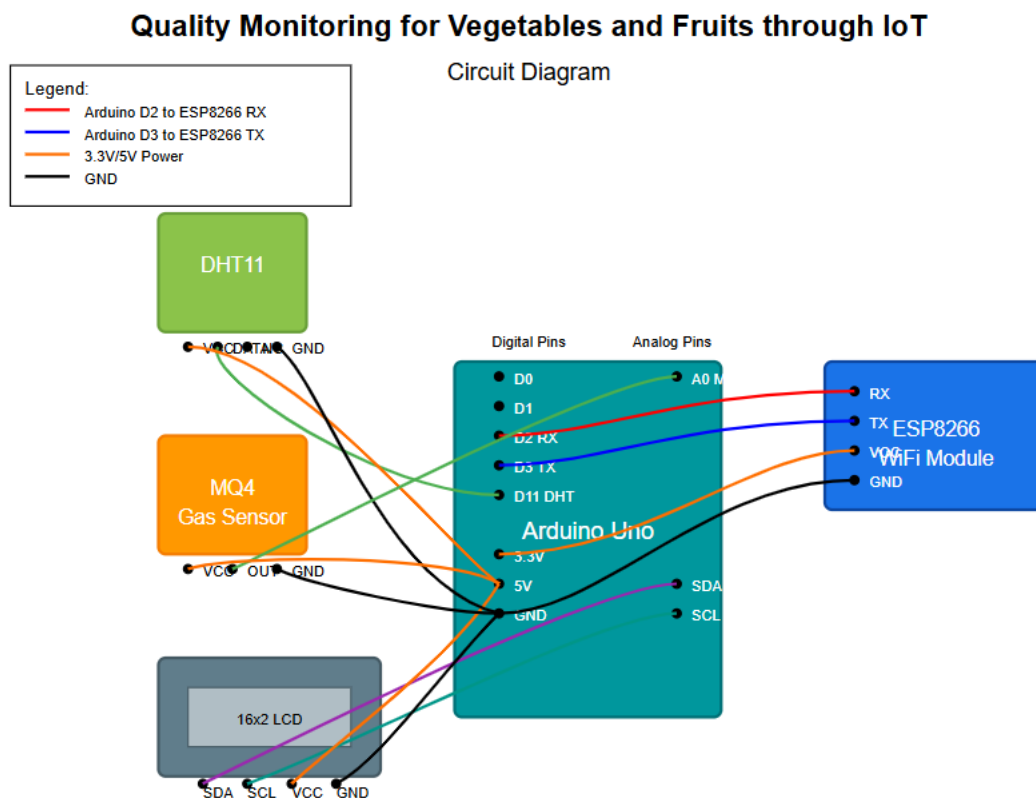


Fig 4.3: circuit Connection Diagram

4.7. Code

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <SoftwareSerial.h>
#include <dht11.h>
#define RX 2
#define TX 3
#define dht_apin 11
#define mq4Pin A0
LiquidCrystal_I2C lcd(0x27, 16, 2);
dht11 dhtObject;
String AP = "Ashu";
String PASS = "Ashu123456";
String API = "AAS9K9EP4JUZZ0C1";
String HOST = "api.thingspeak.com";
String PORT = "80";
int countTrueCommand = 0;
int countTimeCommand = 0;
boolean found = false;
SoftwareSerial esp8266(RX, TX);
void setup() {
  Serial.begin(9600);
  esp8266.begin(115200);
  lcd.init();
  lcd.backlight();
  lcd.setCursor(0, 0);
  lcd.print("Connecting WiFi");
  sendCommand("AT", 5, "OK");
  sendCommand("AT+CWMODE=1", 5, "OK");
  sendCommand("AT+CWJAP=\"" + AP + "\",\"" + PASS + "\"", 20, "OK");
  lcd.clear();
```

```

    lcd.print("WiFi Connected");
    delay(2000);
}

void loop() {
    dhtObject.read(dht_apin);
    int tempValue = dhtObject.temperature;
    int humValue = dhtObject.humidity;
    int gasValue = analogRead(mq4Pin);
    String temp = String(tempValue);
    String hum = String(humValue);
    String gas = String(gasValue);
    String status;
    if (gasValue > 400 || tempValue > 40 || humValue > 80) {
        status = "Spoiled";
    } else {
        status = "Fresh";
    }
    // Display all on LCD
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("T:" + temp + " H:" + hum + " G:" + gas);
    lcd.setCursor(0, 1);
    lcd.print("Status:" + status);
    // Debug logging
    Serial.print("Temperature: "); Serial.println(tempValue);
    Serial.print("Humidity: "); Serial.println(humValue);
    Serial.print("Gas Level: "); Serial.println(gasValue);
    Serial.println("Status: " + status);
    // Construct HTTP GET


---


    String getData = "GET /update?api_key=" + API + "&field1=" + temp + "&field2=" + hum +
"&field3=" + gas;

```

```

// Use single connection mode
sendCommand("AT+CIPMUX=0", 5, "OK");
sendCommand("AT+CIPSTART=\"TCP\", \"" + HOST + "\", " + PORT, 15, "OK");
delay(2000); // Important: allow connection to settle
sendCommand("AT+CIPSEND=" + String(getData.length() + 4), 4, ">");
esp8266.println(getData);
delay(1500);
countTrueCommand++;
sendCommand("AT+CIPCLOSE", 5, "OK");
delay(15000); // wait before next reading}

void sendCommand(String command, int maxTime, char readReplay[]) {
    Serial.print(countTrueCommand);
    Serial.print(". AT Command => ");
    Serial.println(command);
    while (countTimeCommand < (maxTime * 1)) {
        esp8266.println(command);
        if (esp8266.find(readReplay)) {
            found = true;
            break; }
        countTimeCommand++;
    }
    if (found) {
        Serial.println("Command OK");
        countTrueCommand++;
    } else {    Serial.println("Command Failed");
        countTrueCommand = 0;
    }
    countTimeCommand = 0;
    found = false;}

```

CHAPTER 5

RESULTS AND CONCLUSION

5.1 Observations and Graphs:

The project successfully implemented a real-time monitoring system for gas leakage, temperature, and humidity, tailored to ensure the quality and safety of stored vegetables and fruits. The results observed during testing validate the effectiveness of the IoT-based monitoring setup in detecting gas presence and environmental changes, as detailed below.

5.2 Analysis of Sensor Readings:

Real-Time Monitoring Interface

- **Gas Leakage Gauge:** The primary gauge shows a gas level reading greater than 400 ppm, a concentration of ethylene that suggests potential spoilage in perishable items. The scale, with a maximum of 800 ppm, allows users to interpret the gas concentration at a glance and react accordingly. The green color coding represents a safe level, ensuring easy visual monitoring.
- **Temperature Display:** A second gauge presents the temperature in degrees Celsius, with a current reading of 29.5°C. Maintaining an optimal temperature is essential for the freshness of produce, and this real-time display provides immediate feedback for temperature control.
- **Humidity Gauge:** The third gauge shows humidity at 63.1%, which is crucial for preventing excessive moisture or drying of stored produce. Like the gas and temperature gauges, the humidity gauge updates in real-time, providing a continuous monitoring solution.

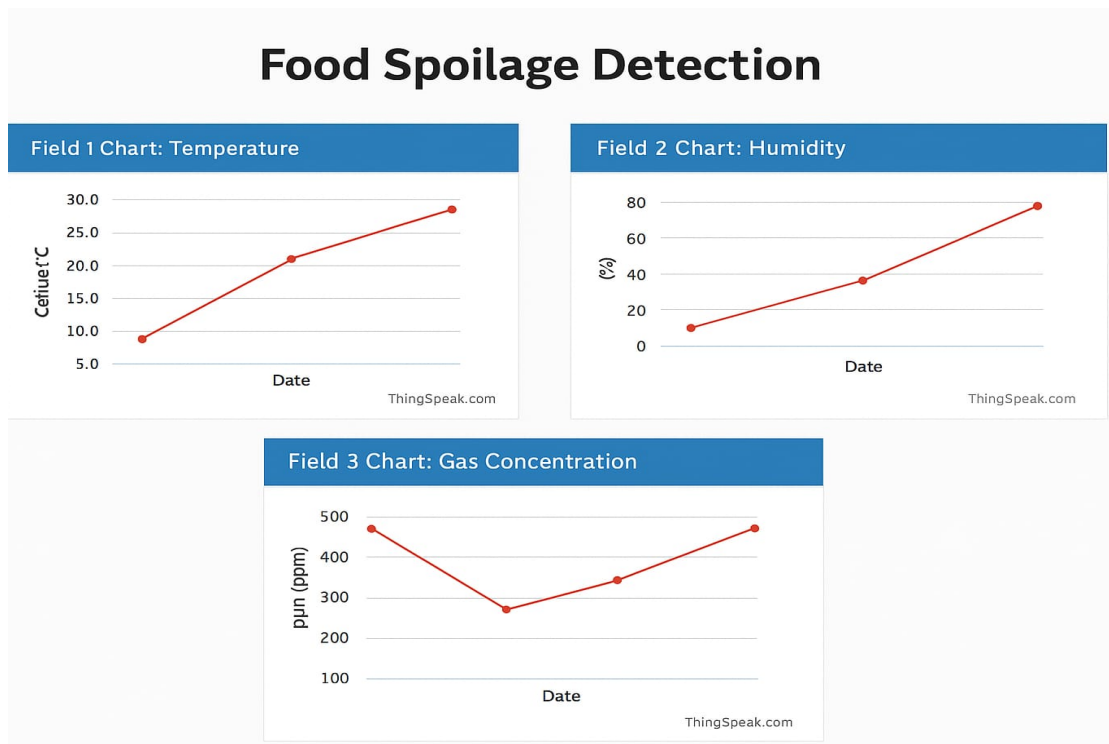


Fig 5.1: Real time monitoring interface



Fig 5.2: LCD display when food is spoiled



Fig 5.3: LCD display when food is fresh

CHAPTER 6

ADVANTAGES AND APPLICATIONS

6.1 Applications

6.1.1. Agricultural Storage and Distribution

This system can be deployed in warehouses, cold storage, and transportation units to monitor temperature, humidity, and gas concentrations, such as ethylene. By continuously monitoring these conditions, the system helps prevent premature spoilage, ensuring that produce remains fresh from farm to market.

6.1.2. Retail and Supermarkets

In supermarkets and retail stores, this monitoring system ensures that fresh fruits and vegetables are displayed for customers by detecting early signs of spoilage. This reduces waste and improves customer satisfaction by offering high-quality produce.

6.2.3. Food Processing and Packaging

The food processing industry can use this system to maintain optimal storage conditions for raw materials and finished products. Monitoring environmental factors can improve product quality, reduce contamination risk, and extend shelf life.

6.2.4. Restaurants and Food Service

In commercial kitchens and restaurants, this monitoring system can help regulate food storage conditions, reducing spoilage and ensuring safety. This application is particularly beneficial for establishments with large storage facilities or those dealing with perishable produce.

6.2.5. Home Use for Smart Refrigerators

The system can be adapted for smart refrigerators in households, providing real-time alerts when temperature or humidity is unfavorable or when produce begins to spoil. This allows families to manage food storage more efficiently, reducing waste.

6.2. Advantages

6.2.1. Enhanced Food Safety

By detecting spoilage indicators such as ethylene gas concentration, temperature, and humidity, the system ensures that only fresh and safe produce reaches consumers, reducing the risk of foodborne illnesses.

6.2.2. Reduction in Food Waste

Early detection of unfavorable conditions allows for timely interventions, preventing large amounts of food from spoiling. This results in a reduction in food waste, which is beneficial for both businesses and the environment.

6.2.3. Real-Time Monitoring and Alerts

The IoT-enabled system provides real-time data on storage conditions, enabling remote monitoring and alerts when certain thresholds are breached. This allows for immediate action to rectify storage conditions and prevent spoilage.

6.2.4. Cost Savings for Businesses

By preserving food quality, the system reduces the costs associated with food waste, spoilage, and potential recalls. This also minimizes the need for frequent quality checks, lowering operational costs.

6.2.6. Scalability and Adaptability

The system is versatile and can be scaled for use in small, medium, or large storage environments. It can be adapted for various produce types by fine-tuning thresholds for gas levels, temperature, and humidity, making it applicable across multiple domains within the food industry.

Bibliography

1. George, A.; Nissen, B.; Broadley, R. Persimmon Nutrition: A Practical Guide to Improving Fruit Quality and Production; Department of Primary Industries, Queensland Horticulture Institute: Indooroopilly, QLD, Australia, 2005.
2. Dong, H.; Kong, X.; Li, W.; Tang, W.; Zhang, D. Effects of plant density and nitrogen and potassium fertilization on cotton yield and uptake of major nutrients in two fields with varying fertility. *Field Crops Res.* 2010, 119, 106–113.
3. Dickinson D. (1972). *Quality Control of Fruits and Vegetables and their Products. Quality Control in the Food Industry*, Vol. 3. London/New York: Academic Press. [A good overview which may be interesting for nonspecialists, too.]

Appendix

1. List of Components Used

S.No	Component	Quantity	Description
1	Arduino Uno	1	Microcontroller board for processing data
2	MQ-4 Gas Sensor	1	Detects methane and other gases
3	DHT11 Sensor	1	Measures temperature and humidity
4	16x2 LCD Display	1	Displays sensor readings
5	ESP8266 WiFi Module	1	Sends data to cloud (ThingSpeak)
6	Breadboard & Jumper Wires	As needed	For circuit connections
7	Power Supply	1	Powers the circuit

2. Circuit Diagram

Refer in the main report for the circuit schematic showing how all components are connected to the Arduino.

3. ThingSpeak Channel Configuration

- Channel Name: Fruit & Vegetable Quality Monitor
- Fields Used:
 - Field 1: Temperature (°C)
 - Field 2: Humidity (%)
 - Field 3: Gas Concentration (ppm)
- Update Interval: 15 seconds
- Alerts: Configured to notify user when gas level exceeds threshold

4. Source Code

The complete Arduino source code is provided in the project repository and attached in the final section of this document.

5. Working Flow Summary

- DHT11 and MQ-4 collect environmental data
- Arduino processes the data.
- LCD displays real-time values.
- ESP8266 transmits data to ThingSpeak.
- ThingSpeak displays the human readable graphs.